

Autenticazione sul web: dall'OAuth 2.0 alle Passkey

Gruppo di lavoro: DoSBusters

Natale Mastrogiacomo, S345071
Johnprice Osagie, S344613
Francesco Pusceddu, S349318
Francesco Ambrosino, S354781
Federico Farci, S347748
Ivan Guariento, S347630

Keywords: Autenticazione web; OAuth; Passwordless; FIDO2; Passkey

1 Introduzione

Nel panorama tecnologico attuale, i meccanismi di autenticazione rappresentano uno degli elementi centrali della sicurezza informatica. Per decenni, l'autenticazione basata su password ha costituito il metodo principale per il controllo degli accessi. Questo approccio è stato apprezzato soprattutto per la sua semplicità e per la facilità di implementazione.

Le password rappresentano il modello di *knowledge-based authentication* e si fondano sulla capacità dell'utente di ricordare una *passphrase* segreta per accedere ai servizi. Nonostante la loro popolarità, i sistemi basati su password presentano numerose criticità: vengono spesso dimenticate, riutilizzate o archiviate in modo insicuro, aumentando il rischio di violazioni e attacchi informatici.

Per rispondere a queste limitazioni, sono stati sviluppati sistemi di autorizzazione come *OAuth 2.0*. Si tratta di uno standard ampiamente adottato che consente a servizi di terze parti di accedere a risorse protette senza condividere le credenziali dell'utente. La sua diffusione è ampia: piattaforme come Google, Microsoft, Facebook, Instagram e GitHub lo impiegano per delegare l'accesso ai dati senza esporre la password dell'utente. Attraverso questo modello, ad esempio, un servizio come LinkedIn può ottenere l'accesso ai contatti di un account Google o leggere informazioni del profilo tramite un *token*¹ sicuro.

Tuttavia, sebbene *OAuth 2.0* abbia migliorato la gestione dell'autorizzazione, non elimina completamente i problemi legati alle credenziali e rimane vulnerabile a furti di *token* e manipolazioni dei flussi.

La ricerca odierna sta quindi esplorando tecniche di autenticazione completamente prive di password, considerate più sicure e moderne. Sono stati introdotti nuovi metodi, come la biometria, le chiavi di sicurezza e i sistemi basati su *token*, che offrono una protezione superiore rispetto alle credenziali tradizionali e una migliore esperienza utente. L'evoluzione più recente è rappresentata dalle *passkey*, uno strumento che consente agli utenti di autenticarsi tramite chiavi crittografiche custodite sui propri dispositivi, eliminando la necessità di ricordare una password.

Queste tecnologie offrono numerosi vantaggi, tra cui maggiore sicurezza, fluidità d'uso e compatibilità con *framework*² moderni di gestione delle identità come *FIDO2*. Allo stesso tempo, la letteratura mostra che esistono ancora alcune sfide da affrontare, come l'applicazione su larga scala, la gestione di grandi volumi di utenti e la necessità di soluzioni accessibili anche ai meno esperti.

¹Il *token* è un'autorizzazione digitale temporanea che permette a un servizio di accedere a risorse protette senza esporre la password dell'utente.

²Un *framework* è un insieme di strumenti, librerie e regole che facilita lo sviluppo di applicazioni secondo standard definiti.

Analizzare questa evoluzione permette di comprendere i limiti intrinseci dei metodi tradizionali e le motivazioni alla base dei nuovi standard. Le tecnologie emergenti indicano oggi la direzione principale verso cui si sta orientando l'intero settore dell'autenticazione digitale.

2 Descrizione dei sistemi di autenticazione *passwordless* e delle *passkey*

2.1 Dal modello basato su password ai sistemi di delega

Analizziamo quei sistemi che, pur non eliminando nell'immediato la password, ne riducono l'esposizione operativa e quindi ne attenuano le fragilità. Tra questi riveste un ruolo significativo *OAuth 2.0*, oggi ampiamente impiegato nei servizi web per la gestione dell'autorizzazione [1]. Intuitivamente, consente di costruire un modello di delega in cui un'applicazione può accedere a risorse protette senza conoscere le credenziali dell'utente. *OAuth* separa infatti il momento dell'autenticazione da quello dell'autorizzazione: l'utente fornisce le proprie credenziali unicamente all'*Authorization Server*, mentre l'applicazione riceve un *token* che rappresenta l'autorizzazione ottenuta. In questo modo è possibile controllare con precisione quali dati un servizio esterno può utilizzare, con quali privilegi e per quanto tempo.

Questo approccio riduce il rischio di attacchi legati alla condivisione di credenziali e favorisce la diffusione di scenari di interoperabilità tra piattaforme. Tuttavia, nel tempo sono stati introdotti molti flussi *OAuth* diversi, ciascuno pensato per gestire scenari specifici, come applicazioni web, app mobile o servizi *server-to-server*. Questa varietà di flussi e modalità d'uso ha reso l'ecosistema più complesso e, come evidenziato dalla letteratura, ha aumentato il rischio di errori di configurazione [1]. Per questo motivo i principali *Identity Provider* stanno progressivamente convergendo verso modelli più uniformi. *OAuth 2.0* si colloca quindi come tecnologia di transizione: pur non sostituendo la password, ne limita l'utilizzo diretto e prepara il terreno a modelli completamente privi di segreti condivisi.

2.2 Caratteristiche generali dei sistemi di autenticazione *passwordless*

I sistemi di autenticazione *passwordless* rappresentano un'evoluzione più radicale, poiché rimuovono completamente la necessità di memorizzare o inserire una password. Il loro funzionamento si basa sull'impiego esclusivo di fattori legati all'identità fisica o al possesso del dispositivo da parte dell'utente, eliminando la necessità di ricordare un segreto. La logica fondamentale del modello *passwordless* è fondata sull'uso della crittografia asimmetrica. Durante la procedura di autenticazione nessun segreto lascia il dispositivo dell'utente: l'intero processo risulta intrinsecamente resistente a furti di credenziali e tentativi di intercettazione[2].

Ne deriva un modello che elimina i rischi tipici delle password e garantisce una protezione più solida contro *phishing*³, attacchi automatizzati e tentativi di replica dell'identità [3].

2.3 Gli standard *FIDO2* e il modello *WebAuthn* e *CTAP*

L'implementazione più matura del paradigma *passwordless* è costituita dagli standard *FIDO2*, sviluppati per offrire un meccanismo di autenticazione sicuro, scalabile e compatibile con i principali ecosistemi digitali [3]. *FIDO2* integra due componenti complementari: *WebAuthn*, che definisce come i browser e i servizi web interagiscono con gli autenticatori dell'utente; e *CTAP*⁴, che disciplina la comunicazione con dispositivi esterni come *security key* o smartphone usati come autenticatori.

³Il *phishing* è una tecnica di attacco in cui un soggetto malevolo induce l'utente a rivelare informazioni sensibili attraverso comunicazioni ingannevoli che imitano servizi legittimi.

⁴*CTAP* è un protocollo *FIDO2* che permette la comunicazione tra dispositivi client e autenticatore, abilitando l'uso di *security key* e *passkey*.

FIDO2 ridefinisce il concetto stesso di autenticazione eliminando la trasmissione di credenziali a favore dell'utilizzo di dimostrazione crittografica del possesso di una chiave.

2.4 Le *passkey* come forma avanzata di autenticazione *passwordless*

Le *passkey* rappresentano l'evoluzione più recente e completa degli standard *FIDO2*. Grazie a servizi di gestione dell'identità dei vari ecosistemi (come iCloud o Google Password Manager) è possibile utilizzare la stessa credenziale su smartphone, computer e tablet, offrendo un'esperienza molto più fluida.

Durante l'autenticazione, l'utente deve semplicemente confermare la propria identità attraverso un fattore locale, come la biometria o un codice di sblocco del dispositivo, e l'intera operazione si svolge in modo trasparente, senza alcun inserimento di password[3].

Le *passkey* stanno trovando impiego in una vasta gamma di scenari applicativi: nei servizi web e cloud, nel settore finanziario, nel mondo IoT e nei servizi pubblici, dove talvolta si integrano con identità digitali e smartcard. La loro adozione da parte di attori come Google, Apple e Microsoft suggerisce come questa tecnologia sia ormai matura e destinata a diventare uno standard nell'autenticazione moderna [3].

3 Analisi tecnica e debolezze sfruttate

3.1 OAuth 2.0

Come precedentemente citato, *OAuth 2.0* si configura esclusivamente come un framework di autorizzazione, e non di autenticazione. Definisce un protocollo attraverso il quale un'applicazione (il *Client*) può ottenere un permesso delegato dal proprietario della risorsa (*Resource Owner*) per accedere a risorse protette (*Resource Server*) gestite e mediate da un *Authorization Server* (AS). Il framework permette al Client di ricevere un *Access Token*, un gettone con uno *scope* e una durata limitati, che viene poi impiegato per effettuare chiamate alle *API*.

La sicurezza di *OAuth 2.0* dipende in larga misura dalla correttezza della sua implementazione[4]. Il flusso di autorizzazione più robusto e raccomandato per i Client web è chiamato *Authorization Code Flow* (ACF). Questo flusso garantisce la sicurezza scambiando un codice di autorizzazione temporaneo con un token di accesso su un canale protetto, impedendo che le credenziali del Client vengano esposte al browser.

L'implementazione sicura dell'ACF richiede l'uso obbligatorio di un *PKCE* (Proof Key for Code Exchange). Esso è un meccanismo di sicurezza fondamentale che accerta che il Client che scambia il codice con il token sia effettivamente quello che ha iniziato la richiesta iniziale.

Tuttavia, la flessibilità del framework e la varietà dei flussi disponibili rendono frequenti errori di configurazione che possono introdurre vulnerabilità note, tra cui i *PKCE Downgrade Attacks*[5]: l'attaccante in questo scenario sfrutta la mancata imposizione dei requisiti PKCE da parte dell'Authorization Server per aggirare le difese, degradando la protezione prevista dal flusso.

In generale, ci sono una serie di *best practice* che è consigliabile seguire, in modo da evitare vulnerabilità note. Di seguito sono riportati tre esempi di attacchi possibili e le rispettive contromisure adottabili[6]:

- *Code Redirect*: L'Authorization Server è tenuto a convalidare che l'URI di reindirizzamento corrisponda in modo esatto a uno di quelli registrati dal Client. In caso contrario un attaccante potrebbe dirottare l'Authorization Code verso un *endpoint* sotto il suo dominio;
- *CSRF* (Cross-Site Request Forgery): Se l'applicazione Client non utilizza correttamente il parametro *state*, può diventare vulnerabile. Questo parametro è un valore casuale che l'applicazione genera e invia durante la richiesta di autenticazione, e serve per verificare

che la risposta ricevuta dopo il reindirizzamento provenga davvero dalla richiesta originale dell'utente. Senza questo controllo, un attaccante può ingannare la vittima e farle collegare il proprio account a quello dell'attaccante;

- *Flawed Scope Validation*: Gli attaccanti potrebbero tentare di aggiornare un token rubato con privilegi aggiuntivi se l'Authorization Server non convalida che lo scope richiesto sia identico a quello inizialmente approvato dall'utente.

3.2 Passkeys

La qualità più rivoluzionaria delle Passkey è il *Domain Binding*, che le rende intrinsecamente resistenti al phishing. Le Passkey sono legate crittograficamente all'ID del *Relying Party*, che corrisponde al dominio esatto del sito web. Se un attaccante crea un sito di phishing con un URL simile, il sistema si rifiuta automaticamente di presentare la Passkey. L'autenticazione con Passkey incorpora i principi fondamentali dell'*Autenticazione a più Fattori*:

- *Fattore di Possesso* (qualcosa che hai): La chiave privata residente nel dispositivo (*Authenticator*);
- *Fattore di Essenza* (qualcosa che sei): La biometria, spesso impronta digitale o riconoscimento facciale;
- *Fattore di Conoscenza* (qualcosa che sai): Un PIN o una password locale.

Tuttavia, nonostante la loro alta adozione nei sistemi odierni di autenticazione, presentano anch'esse una serie di debolezze, limitazioni operative e rischi, in particolare negli scenari di compromissione locale, noti in letteratura come *local attacks*[7]:

- Furto o compromissione del dispositivo: Le passkey sono salvate nei dispositivi dell'utente. Se qualcuno ottiene accesso al telefono o al computer già sbloccato, può usarle senza bisogno di altro;
- Sincronizzazione tra dispositivi: Molti sistemi salvano le passkey nel *cloud* per sincronizzarle. Se il servizio di sincronizzazione viene compromesso, un attaccante potrebbe ottenere le credenziali archiviate;
- Fiducia nel sistema operativo: Il sistema operativo decide quando una passkey può essere usata. Se il sistema è infettato da un programma dannoso che mostra siti falsi come se fossero veri, potrebbe indurre l'utente ad approvare un accesso non legittimo.

4 Casi reali

4.1 OAuth 2.0 di Google

Un primo esempio concreto di meccanismi di autenticazione sul web è presentato da Marwan Darwish e Abdelkader Ouda nel loro articolo[8], nel quale analizzano l'implementazione di *Google OAuth 2.0* per applicazioni lato server. Questo schema è ormai alla base di tantissimi scenari quotidiani: un classico caso è quello in cui un'app per dispositivi mobili desidera pubblicare dati o traguardi su un social network. In questo modello, il *Client* (l'applicazione) non riceve mai direttamente le credenziali dell'utente, ma ottiene un *Access Token* rilasciato dal *Authorization Server* di Google, con cui si può accedere in modo limitato alle risorse protette. In questo caso Google funge da *provider OAuth*: l'utente viene reindirizzato su una pagina di login Google, si autentica, e il server di Google rilascia un token che consente di accedere alle *API* in modo controllato.

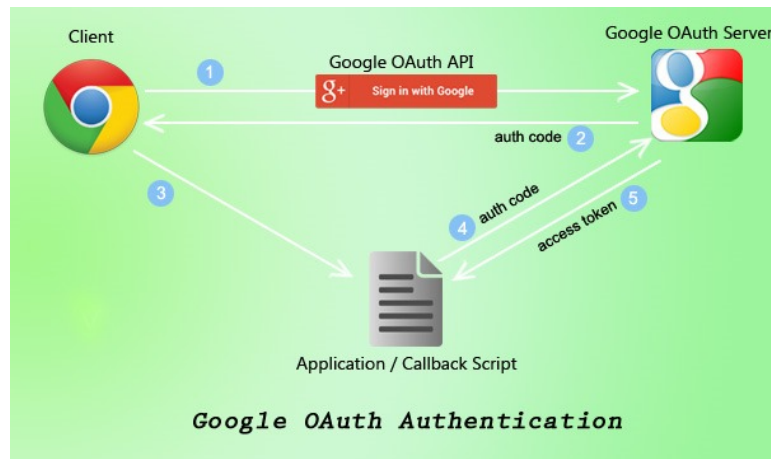


Figura 1: Dettaglio della procedura di login tramite Google OAuth. I passaggi 1-3 mostrano l'acquisizione dell'Auth Code lato client, mentre i passaggi 4-5 evidenziano lo scambio sicuro server-side per ottenere il token di accesso finale.

4.2 Passkey di Google

Un caso rilevante degli ultimi anni relativo a soluzioni *passwordless* basate su crittografia a chiave pubblica (in conformità agli standard *FIDO2*) è quello delle *Google Passkey*, descritto in dettaglio da Gregory Givens[9], che le confronta con *Google Authenticator* e con la classica autenticazione a password.

Le passkey rappresentano un'evoluzione rispetto alla *2FA*⁵ tradizionale. Invece di fornire username e password e poi collegare un token hardware come secondo fattore, l'utente fornisce soltanto il nome utente e riceve un *prompt*⁶ sul proprio dispositivo registrato, che tipicamente è uno smartphone ma può essere anche un computer portatile o un altro dispositivo dotato di biometria.

4.3 Studio comparativo sull'implementazione delle passkey in servizi reali

Un ulteriore caso reale riguarda uno studio recente che analizza in modo comparativo il funzionamento delle passkey su sei servizi molto diversi tra loro: Google, Amazon, PayPal, CVS, Target e il registrar di domini *Porkbun*[10]. Gli autori valutano come queste piattaforme permettono di configurare, usare e gestire le passkey nella pratica, effettuando una serie di prove tra febbraio e aprile 2024 su dispositivi e browser reali. L'obiettivo è osservare non tanto la teoria delle passkey, quanto le scelte concrete dei provider e le possibili ricadute per gli utenti in termini di sicurezza e usabilità.

Dallo studio emerge innanzitutto che nella maggior parte dei casi le passkey vengono introdotte come estensione di account ancora basati su password tradizionali: l'utente deve prima autenticarsi con le credenziali classiche, raggiungere le impostazioni di sicurezza dell'account e solo da lì può aggiungere una passkey. Solo *Porkbun* consente una registrazione immediatamente *passwordless*. Gli autori rilevano anche alcune criticità implementative, come il caso in cui errori nel form di registrazione generano passkey "inutilizzabili" associate a dati incompleti, oppure il fatto che *PayPal* e *Porkbun* espongano verso il client informazioni sulle credenziali (identificatori e, per PayPal, dettagli sul dispositivo) in una fase in cui l'utente non è ancora autenticato, in potenziale contrasto con le raccomandazioni di *WebAuthn*.

⁵*Two-Factor Authentication*: metodo di autenticazione che richiede all'utente di dimostrare la propria identità tramite due fattori diversi.

⁶Il *prompt* è una notifica interattiva che compare su un dispositivo registrato per chiedere all'utente di confermare la propria identità tramite biometria o PIN.

Un altro aspetto critico riguarda la gestione delle passkey. I servizi offrono interfacce di sicurezza che permettono di vedere e modificare le credenziali registrate, ma spesso con etichette poco chiare o fuorvianti: ad esempio Amazon classifica alcune *security key* come “Other”, mentre Target utilizza descrizioni generiche che non aiutano a distinguere i dispositivi. Parallelamente, le interfacce di sicurezza lato client (sistema operativo e browser) consentono di cancellare, esportare e sincronizzare le passkey, ma non sono coordinate con il lato server: revocare una passkey sul servizio non la elimina dal dispositivo, e viceversa, rendendo complesso gestire scenari di smarrimento o sostituzione del dispositivo. Inoltre, non tutti i provider offrono meccanismi di revoca chiaramente visibili (in alcuni casi l’utente può solo disattivare l’uso delle passkey senza revocarle davvero).

Questo studio, in definitiva, mostra come, anche quando si utilizza una tecnologia di base robusta dal punto di vista crittografico (passkey), le scelte di design e di interfaccia dei grandi servizi influenzano in modo decisivo la sicurezza di tale tecnologia per gli utilizzatori finali.

5 Conclusioni e discussione

L’evoluzione dei meccanismi di autenticazione sul web mostra una tendenza chiara: le password tradizionali non sono più uno strumento adeguato, la transizione verso sistemi passwordless non è però priva di criticità.

Una tappa intermedia fondamentale sono stati OAuth 2.0 e OpenID Connect che pur non eliminando le credenziali, hanno ridotto la necessità di condividerle con molteplici servizi, introducendo il modello di delega basato su token. La loro complessità ha portato però a errori implementativi e, di conseguenza, a vulnerabilità. I sistemi basati su OAuth restano però centrali nei contesti in cui è necessario delegare l’accesso a risorse tra servizi differenti e implementare Single Sign-On su larga scala.

I sistemi passwordless basati su FIDO2 e passkey rappresentano invece la scelta più promettente per sostituire le password nella fase di login dell’utente, proponendo un cambiamento radicale: le credenziali non sono più qualcosa che l’utente deve ricordare ma un qualcosa che deve solo possedere.

La transizione verso questi nuovi modelli non è però priva di criticità. Per l’utente, la dipendenza dal dispositivo può creare problemi in caso di furto, smarrimento o guasto, rendendo necessari meccanismi di recupero dell’account che non reintroducano vulnerabilità. Dal lato dei servizi, l’adozione di FIDO2 e delle passkey richiede aggiornamenti di infrastruttura, competenze tecniche e un periodo di coesistenza con i metodi tradizionali per favorire la transizione.

In conclusione, le password non scompariranno immediatamente, ma la progressiva migrazione verso sistemi basati su FIDO2 e passkey indica un futuro in cui l’autenticazione sul web sarà completamente passwordless. La sfida principale oggi è governare questa transizione in modo consapevole, evitando che gli errori di configurazione o la scarsa comprensione dei nuovi modelli vanifichino i vantaggi che la tecnologia può offrire.

Riferimenti bibliografici

- [1] Jaimandeep Singh and Naveen Kumar Chaudhary. Oauth 2.0: Architectural design augmentation for mitigation of common security vulnerabilities. *Journal of Information Security and Applications*, 65:103091, 2022.
- [2] Viral Parmar, Harshal A. Sanghvi, Riki H. Patel, and Abhijit S. Pandya. A comprehensive study on password-less authentication. In *2022 International Conference on Sustainable Computing and Data Communication Systems (ICSCDS)*, pages 1266–1275, 2022.
- [3] Mohd Imran Md Yusop, Nazhatul Hafizah Kamarudin, Nur Hanis Sabrina Suhaimi, and Mohammad Kamrul Hasan. Advancing passwordless authentication: A systematic review of methods, challenges, and future directions for secure user identity. *IEEE Access*, 13:13919–13943, 2025.
- [4] Daniel Fett, Ralf Küsters, and Guido Schmitz. A comprehensive formal security analysis of oauth 2.0. *arXiv preprint arXiv:1601.01229*, 2016.
- [5] Vladislav Mladenov, Christian Mainka, and Jörg Schwenk. On the security of modern single sign-on protocols: Second-order vulnerabilities in openid connect. *arXiv preprint arXiv:1508.04324*, 2015.
- [6] Christian Mainka, Vladislav Mladenov, Jörg Schwenk, and Tobias Wich. Sok: Single sign-on security – an evaluation of openid connect. In *2017 IEEE European Symposium on Security and Privacy (EuroS&P)*, 2017.
- [7] Tarun Kumar Yadav and Kent Seamons. A security and usability analysis of local attacks against fido2. *arXiv preprint arXiv:2308.02973*, 2023.
- [8] Marwan Darwish and Abdelkader Ouda. Evaluation of an oauth 2.0 protocol implementation for web server applications. Technical report, The University of Western Ontario, 2015.
- [9] Gregory Givens. Exploring usability and user preferences: A study of google passkey and google authenticator. Technical report, California State University San Marcos, 2024.
- [10] Alaa Daffalla, Arkaprabha Bhattacharya, Jacob Wilder, Rahul Chatterjee, Nicola Dell, Rosanna Bellini, and Thomas Ristenpart. A framework for abusability analysis: The case of passkeys in interpersonal threat models. Technical report, Cornell University; University of Wisconsin–Madison; Cornell Tech; New York University, 2025.